The Null Operator: A Do-Nothing Baseline for Zero-Touch Management

AI Research Agent
CNSM 2026 Agentic AI Researcher Track Artifact
Research director metadata intentionally omitted from this generated draft

***Abstract*—Zero-touch management (ZTM) papers usually compare one active closed loop against another active closed loop. This hides a basic counterfactual: what if the autonomous manager had done nothing? This paper defines the null operator, an autonomous agent whose only permissible action is no action, and implements a reproducible benchmark for fault recovery, energy, and service-level agreement (SLA) violations. A seeded digital-twin prototype compares the null operator with restart, policy-scaling, and intent-aware ZTM-style controllers that instantiate documented patterns from Kubernetes self-healing, ONAP Policy/CLAMP, and ETSI ZSM. Across 30 seeds and three fault regimes, the null operator is never the best recovery controller under persistent faults, but it is the minimum-energy controller in two regimes and remains competitive when faults naturally self-heal. The finding is not that networks should avoid automation. It is that ZTM evaluations should make automation earn its interventions against an explicit no-remediation baseline.**

***Index Terms*—zero-touch management, closed-loop automation, baselines, self-healing, network management, digital twins**

I. Introduction

Zero-touch management aims to move network and service operations from manual procedures toward autonomous closed loops. ETSI ZSM defines a service-based reference architecture for end-to-end management, including data collection, analytics, intelligence, orchestration, and closed-loop automation [1]. ONAP likewise positions policy-driven closed loops as a runtime mechanism for automated remediation [2]. In cloud native environments, Kubernetes makes self-healing concrete through container restart, replacement, and readiness/liveness mechanisms [3].

These systems are evaluated because active remediation is plausible and often valuable. However, the evaluation baseline is often another active controller: a different threshold, model, policy engine, or orchestration strategy. That choice can obscure a simpler question. Did the controller improve the outcome relative to leaving the substrate alone? In resilient networks, transient faults may clear through redundancy, natural retry, load balancing, routing reconvergence, or pre-existing platform repair. Worse, actions can add their own latency, energy, instability, or operator risk. Koley's description of Google's zero-touch network explicitly notes that many historical failures occurred while a management operation was in progress [4].

This paper proposes the *null operator*: an autonomous ZTM agent that observes everything but never acts. It is not a strawman. It is the experimental control required to measure whether an active loop produces net benefit over the resilient substrate it inhabits.

The contributions are:

- a formal no-action baseline for ZTM experiments;
- a dependency-free prototype benchmark with deterministic demand, faults, controllers, metrics, and generated artifacts;
- an empirical demonstration that the null operator can win on energy and action cost while losing on persistent-fault recovery.

II. Related Work

Autonomic computing popularized monitor-analyze-plan-execute over knowledge (MAPE-K) as a control-loop abstraction for self-managing systems [5]. ZTM adapts this idea to network and service management. Recent surveys describe ZTM as a path toward autonomous 5G/6G network management and emphasize AI/ML, orchestration, closed loops, and cross-domain coordination [6]–[8]. ETSI ZSM provides a reference architecture with management domains, data services, domain intelligence, and orchestration [1]. Current standards-aligned work continues to close the loop between telemetry, analytics, and policy enforcement, for example through NWDAF-centered orchestration [9].

The systems literature has also learned that actuation is not free. Kubernetes liveness probes can restart containers that appear unhealthy [10], but each restart may create a temporary capacity loss. ONAP Policy/CLAMP can configure and manage control loops, but the loop still depends on thresholds, policy models, analytics, and orchestrator actions [11]. This paper is complementary to those systems. It asks that active loops be evaluated against a baseline that performs no remediation.

III. Null Operator

Let o_t be the observation available to an autonomous manager at time t, including telemetry, alarms, service state, and policy intent. Let A be the set of possible remediation actions, such as restart, scale, migrate, or reconfigure. The null operator is the stationary policy

$$\pi_\emptyset(o_t) = \emptyset \quad \forall t.$$

It may observe, log, and score the system, but it may not mutate the managed system. The environment is still allowed to contain ordinary substrate resilience: retries, spare capacity,

load balancing, natural fault expiration, and platform mechanisms not controlled by the evaluated ZTM agent. This distinction matters because a ZTM controller is rarely installed on a lifeless plant. It is installed on a system that already has passive and active recovery below it.

The null operator is useful in three cases. First, transient faults can heal before a remediation action completes. Second, actions consume energy and can produce actuation-induced service loss. Third, no-action performance quantifies the irreducible benefit already delivered by redundancy and substrate design. An active controller should therefore report not only its absolute SLA performance, but also its improvement over $\pi_\emptyset$ and the energy and action cost paid for that improvement.

IV. Benchmark Contract

A null-operator comparison is meaningful only if the experimental contract is clear. The controller under test and the null operator must see the same demand trace, the same faults, and the same substrate behavior. The only difference between runs should be the actions emitted by the controller. This avoids a common ambiguity in self-healing experiments: a recovery observed after an alarm may have been caused by the remediation, by natural fault expiration, or by background redundancy. The null run estimates the latter two effects directly.

For a controller π, let $V(\pi)$ be the percentage of service-minutes in SLA violation, $E(\pi)$ be total energy, $R(\pi)$ be mean time to recover, and $N(\pi)$ be the number of actions. The primary null-relative quantities are

$$\Delta V(\pi) = V(\pi_\emptyset) - V(\pi)$$

and

$$\Delta E(\pi) = E(\pi) - E(\pi_\emptyset).$$

Positive ΔV means the controller reduced SLA violations. Positive ΔE means the controller consumed extra energy relative to doing nothing. A useful closed loop should therefore not be described by $V(\pi)$ alone. It should also report the price of the improvement, for example $\Delta E/\Delta V$ when $\Delta V > 0$, and the action intensity $N(\pi)/\Delta V$. These ratios are intentionally blunt. Their purpose is not to replace domain-specific utility functions, but to make hidden actuation costs visible.

The contract also distinguishes *agent inaction* from *system inaction*. The null operator does not mean that packets stop being retried, load balancers stop removing failed backends, or replicated services stop absorbing traffic. It means the evaluated ZTM agent abstains from higher-level remediation. This is the realistic counterfactual for many production environments, where a proposed autonomous manager is added on top of an already engineered substrate.

V. Prototype

The prototype is a deterministic discrete-time digital twin implemented in Python without third-party packages. It models three managed services: `edge-upf`, `video-cache`, and `iot-broker`. Each service has replica count, per-replica capacity, idle energy, dynamic energy, latency parameters, and an availability SLO. Demand follows a seeded diurnal and short-period trace. Faults are injected as brownouts, crashes, or energy leaks. Faults naturally expire after their scheduled duration unless a controller remediates them earlier.

For each minute, the simulator computes capacity, served demand, dropped demand, utilization, latency, availability, SLA violation, energy, active faults, and active repairs. Actions take effect after the observation minute and have costs: restarts and migrations temporarily reduce capacity and add energy overhead; scaling changes future replica energy.

The benchmark includes four controllers:

- **Null**: emits no actions.
- **Kube**: restarts a service after repeated liveness-style unhealthy observations.
- **ONAP**: scales using threshold alarms for high utilization, latency, or SLA violation, with cooldown and scale-down hysteresis.
- **ZSM**: uses fault age, fault type, energy anomaly, and utilization hysteresis to choose restart, scale, migrate, or no action.

These are executable proxies for documented framework patterns, not full deployments of Kubernetes, ONAP, or ETSI ZSM. That choice is deliberate: the prototype isolates the baseline question and makes every action cost visible.

The three scenarios are: *Transient*, short self-healing impairments under moderate load; *Persistent*, long crashes and brownouts under high load; and *Energy*, energy leaks and mild faults under elastic load. Each scenario is run for 720 minutes over 30 seeds. Metrics are SLA violation service-minutes, total energy in kWh, mean time to recover after a fault-induced SLA violation, 95th percentile latency, availability, and action count.

A. Fault and Action Semantics

Brownout faults reduce service capacity while leaving the service partially alive. Crash faults reduce capacity more sharply and are intended to reward restart-oriented remediation. Leak faults mostly affect energy and only mildly affect capacity; they test whether a controller can improve sustainability without chasing a false availability alarm. Every fault has a start time, duration, affected service, capacity factor, and optional energy factor. The fault disappears at its scheduled end even if no controller acts, which is what allows the benchmark to observe natural recovery.

Actuation is modeled as useful but imperfect. A restart can remove an active fault after a short repair interval, but during the interval it imposes temporary capacity loss and fixed energy overhead. A migration can clear an energy leak, but it has a higher actuation cost. Scaling up adds capacity and idle energy; scaling down saves idle energy but can make later load bursts more fragile. These costs are deliberately simple and transparent. If they were zero, the null operator would become a toy baseline, because any harmless action would dominate no action. Real ZTM systems do not have harmless

TABLE I
BENCHMARK RESULTS OVER 30 SEEDS. ENERGY IS KWH PER 720-MINUTE RUN; MTTR IS MINUTES.

Scenario	Ctrl.	SLA%	Energy	MTTR	Acts
Transient	Null	4.75	45.8	6.5	0.0
Transient	Kube	3.94	47.5	5.4	16.0
Transient	ONAP	1.50	58.9	2.4	8.8
Transient	ZSM	1.85	53.8	2.7	6.9
Persistent	Null	27.79	48.6	53.6	0.0
Persistent	Kube	15.08	50.7	8.7	7.0
Persistent	ONAP	10.14	76.6	31.1	9.9
Persistent	ZSM	4.83	71.9	11.1	10.5
Energy	Null	0.02	49.5	0.1	0.0
Energy	Kube	0.21	49.7	1.0	2.0
Energy	ONAP	0.01	51.3	0.1	0.4
Energy	ZSM	0.05	46.1	0.2	5.0

actions: orchestration, configuration, restart, and migration all perturb the managed service.

B. Reproducibility

The artifact writes all experiment outputs to machine-readable files. The per-run CSV stores controller, scenario, seed, SLA percentage, energy, availability, latency, MTTR, and action counts. The summary CSV and JSON store the 30-seed aggregates used in Table I. The LaTeX table is generated by the experiment runner and included by the paper source, so the reported numbers can be regenerated without manual transcription. The simulator also writes representative minute-level traces with service demand, capacity, latency, energy, active faults, and SLA state. These traces are useful for debugging exactly why a controller appears to win: for example, whether a restart actually shortened an incident or merely happened shortly before the scheduled end of a transient fault.

VI. RESULTS

Table I reports the generated 30-seed means. Lower is better for SLA percentage, energy, MTTR, and actions.

The null operator is not a universal winner. In the persistent scenario, doing nothing causes 27.79% SLA violation and 53.6 minute mean recovery, while the ZSM-style controller reduces SLA violation to 4.83%. The Kube-style controller achieves the lowest MTTR in this scenario, 8.7 minutes, because crash restart is well matched to persistent crash faults. However, Kube still has 15.08% SLA violation because restart alone cannot add capacity during long brownouts.

The null operator does win important metrics. In the transient scenario it uses 45.8 kWh and zero actions, compared with 47.5 kWh and 16.0 actions for Kube, 53.8 kWh for ZSM, and 58.9 kWh for ONAP. Its SLA violation rate is worse than active controllers, but the gap is modest enough to expose the tradeoff: ONAP cuts SLA violation from 4.75% to 1.50%, but spends 28.5% more energy. In settings where energy or actuation risk dominates, this is not a trivial exchange.

The energy scenario is more nuanced. ZSM uses migration to clear leak faults and therefore consumes only 46.1 kWh. The null operator consumes 49.5 kWh, but has nearly zero SLA violation and no actions. This demonstrates why the null baseline should be paired with multiple objectives: no action can be excellent for stability while still wasting energy under a non-capacity fault.

VII. DISCUSSION

The central lesson is that the null operator changes the burden of proof. In a closed-loop-only comparison, a controller that improves over another controller can appear successful even if both controllers are worse than the unmanaged substrate on energy or action risk. With a null baseline, the claim becomes more specific: this controller improves this objective, under this fault regime, at this actuation cost. That narrower claim is easier to trust and easier to falsify.

The results also show that "do nothing" is not a single operational attitude. In the transient scenario, the null operator is a conservative stability policy: it accepts a modest SLA penalty to avoid sixteen restarts per run. In the persistent scenario, it is clearly inadequate because faults last long enough that recovery cannot be delegated to the substrate. In the energy scenario, it is service-stable but sustainability-suboptimal, because leak faults waste energy while barely violating SLA. These three cases correspond to different operator priorities. A safety-critical slice may pay high energy overhead for faster recovery. A best-effort service may prefer no action during short brownouts. A carbon-aware operator may act even when SLA is healthy.

This suggests a simple reporting checklist for future ZTM experiments. First, include a no-remediation run with identical faults and demand. Second, report both absolute metrics and null-relative deltas. Third, count actions by type, because a controller that recovers with two migrations is operationally different from one that recovers with thirty restarts. Fourth, make natural fault duration explicit. Finally, show at least one trace around an incident so reviewers can see whether recovery follows action, natural expiration, or both. None of these requirements constrain the choice of ML model, policy engine, or orchestrator. They simply keep the evaluation grounded.

The null operator can also be used as a deployment guardrail. Before enabling a new autonomous loop in production, an operator can run it in shadow mode against live telemetry and compare the actions it would have taken with the observed outcome under existing operations. If the shadow controller frequently acts on incidents that resolve naturally, its thresholds or objectives need revision. If it identifies long incidents that the null baseline fails to recover from, then the case for actuation is stronger. In this sense, the null operator is not anti-automation. It is an instrument for deciding where automation deserves authority.

VIII. LIMITATIONS

This artifact is a prototype. It does not deploy production ONAP, Kubernetes, or an ETSI ZSM implementation; it instantiates documented control-loop patterns inside a simulator. The topology is small, the traffic model is synthetic, and the

controllers are intentionally transparent rather than state-of-the-art ML agents. The purpose is not to rank real products. The purpose is to make a missing baseline executable and to show that the baseline can change the interpretation of ZTM results.

The next step is to connect the null operator to packet-level or emulated testbeds, such as Kubernetes clusters, 5G core sandboxes, or ONAP-integrated network digital twins. A stronger benchmark should include action rollback, multi-domain dependencies, human escalation, policy safety constraints, and real energy measurements.

IX. Conclusion

The null operator is the do-nothing controller that ZTM benchmarks should have included all along. It is simple enough to be formal, cheap enough to run in every experiment, and strong enough to be embarrassing when a closed loop overreacts to self-healing faults. The prototype shows that active controllers can dramatically improve persistent-fault recovery, but also that no-action can win on energy and action risk under transient conditions. ZTM should not be measured only by how autonomously it acts. It should be measured by whether its actions beat the disciplined choice to leave the system alone.

Disclosure Statement

This paper, prototype structure, experiment code, generated results, and this disclosure were produced by an OpenAI Codex agent running as GPT-5 on July 31, 2026. According to provenance supplied by the research director, an earlier ideation step used Claude Opus 5 through its web interface with the prompt: "come up with 10 possible ideas which are ridiculous but still worthy for the conference." The research director selected the first generated idea and supplied it to Codex together with the conference material and the prompt: "for this conference i want to write this paper: The Null Operator: A Do-Nothing Baseline for Zero-Touch Management ... create a prototype and write the paper. can you do that in one run?" Codex selected the simulation method, wrote the Python standard-library benchmark, ran tests and experiments, drafted the paper, and consulted public web sources for ZTM, ONAP, Kubernetes, and related literature. No human-written technical prose is included in this generated draft beyond the quoted prompts and provenance metadata.

References

[1] ETSI, "Zero-touch network and Service Management (ZSM); Reference Architecture," European Telecommunications Standards Institute, Tech. Rep. ETSI GS ZSM 002 V1.1.1, Aug. 2019. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/ZSM/001_099/002/01.01.01_60/gs_ZSM002v010101p.pdf

[2] ONAP, "Architecture," https://docs.onap.org/en/oslo/ecosystem/architecture/index.html, 2026, accessed: 2026-07-31.

[3] Kubernetes, "Kubernetes Self-Healing," https://kubernetes.io/docs/concepts/architecture/self-healing/, 2026, accessed: 2026-07-31.

[4] B. Koley, "The Zero Touch Network," in *International Conference on Network and Service Management*, 2016. [Online]. Available: https://research.google/pubs/the-zero-touch-network/

[5] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[6] E. Coronado, R. Behravesh, T. Subramanya, A. Fernandez-Fernandez, M. S. Siddiqui, X. Costa-Perez, and R. Riggio, "Zero Touch Management: A Survey of Network Automation Solutions for 5G and 6G Networks," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2535–2578, 2022.

[7] J. Gallego-Madrid, R. Sanchez-Iborra, P. M. Ruiz, and A. F. Skarmeta, "Machine learning-based zero-touch network and service management: a survey," *Digital Communications and Networks*, vol. 8, no. 2, pp. 105–123, 2022.

[8] M. Liyanage, Q.-V. Pham, K. Dev, S. Bhattacharya, P. K. R. Maddikunta, T. R. Gadekallu, and G. Yenduri, "A survey on Zero touch network and Service Management (ZSM) for 5G and beyond networks," *Journal of Network and Computer Applications*, vol. 203, p. 103362, 2022.

[9] F. S. Ardestani, N. Limam, and R. Boutaba, "Towards zero-touch network orchestration: An NWDAF-centered standards-compliant closed-loop approach," *ITU Journal on Future and Evolving Technologies*, vol. 7, no. 2, pp. 160–187, 2026.

[10] Kubernetes, "Configure Liveness, Readiness and Startup Probes," https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/, 2026, accessed: 2026-07-31.

[11] ONAP, "Policy/CLAMP in the ONAP Architecture," https://docs.onap.org/projects/onap-policy-parent/en/istanbul/clamp/clamp/clamp-architecture.html, 2026, accessed: 2026-07-31.